JOB PORTAL SYTEM

A PROJECT REPORT

Submitted by

Keshav Chawla (24BCS12851)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Chandigarh University

INTRODUCTION:-

The **Job Portal System** is a web-based application designed to simplify the recruitment process by connecting job seekers and employers on a single digital platform. In the modern era, traditional job search methods such as newspaper advertisements and manual applications have become inefficient and time-consuming. With the rapid growth of internet technologies, online job portals have become essential tools for recruitment and employment. This project aims to develop a full-stack Job Portal System using modern technologies such as **React** for frontend development, **Spring Boot** for backend services, and MongoDB for database management. The system allows users to register, log in, search for job listings, and apply for jobs, while employers can post job openings and manage applications efficiently. The system provides a user-friendly interface that ensures easy navigation and accessibility. It focuses on improving the overall experience of both job seekers and recruiters by providing real-time job updates and streamlined application processes. Security is a major concern in such applications; therefore, **JWT (JSON Web Token)** authentication is implemented to ensure secure communication and data protection. This project also emphasizes scalability and flexibility. The use of **MongoDB** allows dynamic data handling, making it suitable for applications that require frequent updates. The modular design ensures that new features such as AI-based recommendations and **resume parsing** can be added in the future. Overall, the Job Portal System serves as an effective solution for modern recruitment challenges. It reduces manual effort, saves time, and improves efficiency for both employers and job seekers.

OBJECTIVES:-

The main objectives of this project are:

- To develop a **basic full-stack web application**
- To understand **client-server architecture**
- To implement **REST APIs using Spring-boot.**
- To build a dynamic user interface using **React**
- To implement **search and filtering functionality**
- To simulate real-world job portal behavior

FUNCTIONAL REQUIREMENTS :-

User Functional Requirements

1. View Job Listings

- The system should display all available jobs.
- Each job should be shown as a **card** containing:
    - Job title
    - Company name
    - Location
    - Category

2. Search Jobs

- User should be able to search jobs using keywords.
- Search should work on:
    - Job title
    - Company name
- The system filters results dynamically.

3. Filter Jobs by Category

- Users should be able to filter jobs based on categories such as:
    - IT
    - Marketing
    - Finance
- Only matching jobs should be displayed.

4. View Job Details

- User should be able to click on a job and view:
    - Full description
    - Requirements
    - Salary
    - Location

5. Apply for Job

- User should be able to fill an application form.
- Form fields:
    - Name
    - Email
    - Phone
    - Resume (text input)

6. Application Submission

- After submission:
    - System should validate input
    - Send data to backend
    - Show success message

System Functional Requirements

1. API to Fetch All Jobs

- Endpoint: GET /api/jobs
- Returns all job listings

2. API to Fetch Job by ID

- Endpoint: GET /api/jobs/:id
- Returns specific job details

3. API to Submit Application

- Endpoint: POST /api/apply
- Accepts form data
- Sends success response

4. Data Handling

- Job data stored in jobs.js
- Data is sent in JSON format

4. SYSTEM WORKFLOW:-

Step 1: User Access

- User opens the website
- Home page is displayed

Step 2: Navigation to Jobs Page

- User clicks “Explore Jobs”
- React Router loads Jobs page

Step 3: Fetch Jobs from Backend

- Frontend sends request:

GET /api/jobs

- Backend reads data from jobs.js
- Sends job list as response

Step 4: Display Jobs

- React stores jobs in state
- Displays them using JobCard component

Step 5: Search and Filter

- User types search keyword
- React filters job array using .filter()
- UI updates dynamically

Step 6: View Job Details

- User clicks “View Details”
- URL changes to /jobs/:id
- Frontend sends:

GET /api/jobs/:id

- Backend returns specific job

Step 7: Apply for Job

- User clicks “Apply Now”
- Form page opens
- User enters details

Step 8: Submit Application

- Frontend sends:

POST /api/apply

- Backend receives data
- Validates fields
- Logs application

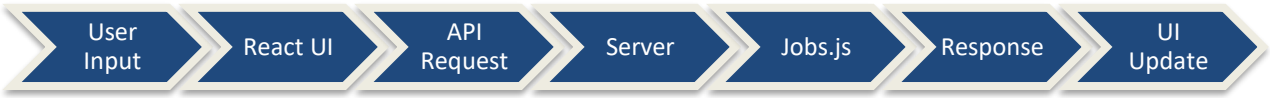
Step 9: Confirmation

- Backend sends success message
- Frontend displays confirmation

5. DATA FLOW DIAGRAM (DFD):-

Level 0:-

Level 1:-



6. SYSTEM ARCHITECTURE:-

The system follows **Client-Server Architecture**:

Frontend (Client)

- React.js
- Handles UI and user interaction

Backend (Server)

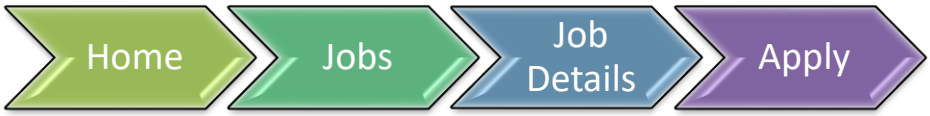
- Handles API and data processing
- Spring-boot

Data Layer

- JavaScript array (jobs.js)

7. SYSTEM LAYOUT:-

Pages Structure:-



8. LIMITATIONS:-

1. No Database
- Data is not stored permanently
- All jobs are hardcoded

2. No Authentication
- No login/signup system
- Anyone can apply

3. No Data Persistence
- Applications are not saved
- Data is lost on server restart

4. No Resume Upload
- Resume is only text input
- No file handling

5. Limited Scalability
- Cannot handle large number of users
- Not suitable for real-world deployment

6. No Admin Panel
- Cannot add/edit/delete jobs dynamically

9. FUTURE ENHANCEMENTS:-
- Add MongoDB database
- Implement login system
- Add admin dashboard
- Enable resume upload
- Improve UI/UX
- Deploy application online

10. CONCLUSION:-

The Job Portal System developed in this project successfully demonstrates the design and implementation of a full-stack web application that connects job seekers and employers on a unified platform. The system addresses the limitations of traditional recruitment methods by providing a digital solution that is efficient, scalable, and user-friendly. By integrating modern technologies such as React for the frontend, Spring Boot for backend processing, and MongoDB for data storage, the project achieves a robust and flexible architecture. The application allows job seekers to register, log in, browse available job listings, and apply for positions in a streamlined manner. Similarly, employers can post job openings, manage listings, and review applications efficiently. The implementation of JWT-based authentication ensures secure access to system resources and protects sensitive user data.

The modular structure of the system enhances maintainability and allows easy integration of additional features in the future. Throughout the development process, various phases such as system analysis, design, implementation, and testing were carefully executed to ensure that the system meets functional and non-functional requirements. The testing phase confirmed that all modules operate correctly and that the system provides reliable performance under different conditions. This project not only fulfills its objective of creating a functional job portal but also provides practical exposure to full-stack development concepts, including RESTful APIs, database management, and secure authentication. It serves as a solid foundation for developing more advanced recruitment platforms with enhanced capabilities.